



DL-HIDS: deep learning-based host intrusion detection system using system calls-to-image for containerized cloud environment

Nidhi Joraviya¹ · Bhavesh N. Gohil¹ · Udai Pratap Rao²

Accepted: 4 January 2024 / Published online: 9 February 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

In the rapidly evolving IT industry, containerization has introduced new security challenges including cloud data breaches. DL-HIDS explores the application of Deep Learning (DL) techniques for detecting such attacks. Various system call-based features, including the sequence, frequency, and metadata of system calls, as well as images, derived from these calls were explored. While using images as features is effective for DL models, determining the optimal image feature size can be challenging and requires extensive experimentation. The existing approach uses pre-trained Convolutional Neural Networks (CNNs) that incorporate system call parameters with metadata that are redundant resulting in a low detection rate. To address these limitations, we employ a deep CNN that takes images generated from system call logs as input. Our experimentation involves varying image size, system call parameters, and CNN architecture using the Leipzig Intrusion Detection Data-Set-2019 dataset containing recent containerized cloud environment attack data. Our results demonstrate improvement over state-of-the-art methods toward accuracy, precision, recall, F1 score, and false-positive rate.

Keywords Cloud computing · Containerized environment · Deep learning · Intrusion detection system · System call analysis

Bhavesh N. Gohil and Udai Pratap Rao have contributed equally to this work.

✉ Nidhi Joraviya
D20CO002@coed.svnit.ac.in

Bhavesh N. Gohil
bng@coed.svnit.ac.in

Udai Pratap Rao
udai.cs@nitp.ac.in

¹ Department of Computer Science and Engineering, Sardar Vallabhbhai National Institute of Technology, Icchanath, Surat, Gujarat 395007, India

² Department of Computer Science and Engineering, National Institute of Technology, Patna, Bihar 800005, India

1 Introduction

The rapid evolution of cloud computing has brought about significant advancements through the integration of container technology. Containers offer Operating System (OS)-level virtualization that allows multiple isolated applications or systems to operate on a single kernel of the host OS in a cloud environment [1]. This approach has gained popularity over traditional virtualization methods due to its efficiency, reliability, and facilitation of Continuous Integration/Continuous Deployment (CI/CD) processes. Recognizing their potential, organizations are increasingly adopting containerized environments. Gartner projects the global market for container maintenance and provision to reach \$944 million by 2024 [2]. Additionally, it is expected that 47% of IT service providers will implement container technology in their operations [3].

Containers are lightweight and standalone executable packages that include everything necessary to run cloud applications, such as code, system tools, libraries, and settings. They are broadly categorized into application containers and lightweight OS virtualization. The usage of containers primarily revolves around hosting applications or services of an application. A Container Engine is the software that handles user requests, fetches images, and executes containers. These images are often stored in public repositories, pulled by a daemon process known as the container runtime or engine, and utilized as microservices in the applications running on the host OS. The workflow of the container running on the host OS is shown in Fig. 1.

Despite their advantages, containers pose significant security risks. They can be vulnerable to attacks from fellow containers or the host environment. Incidents like the Man-In-The-Middle attack through CVE-2020-8554 [4], Siloscape malware [5], and crypto-hijacking via CVE-2018-15664 [6] underscore the vulnerabilities in containerized cloud environments. Moreover, misconfigured Kubernetes clusters and

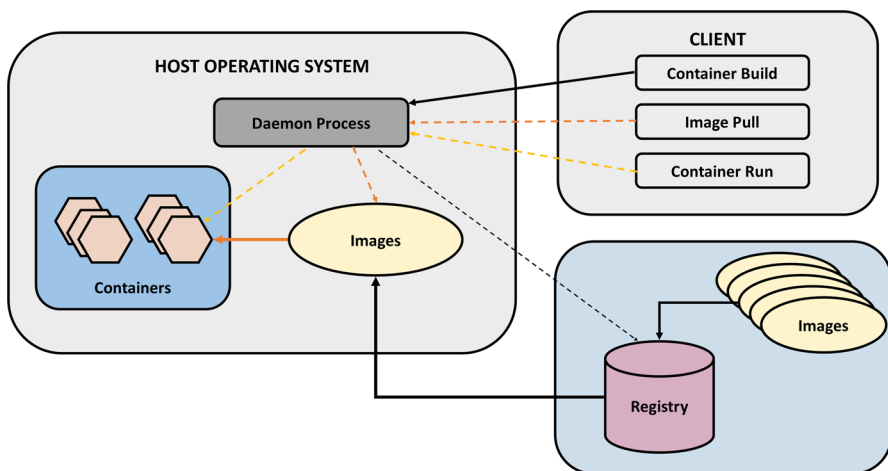


Fig. 1 Working of containers in real-time environment

outages in major cloud services like AWS, which affected companies like Samsung and Expedia, highlight the need for robust security measures [7]. Containerized platforms of Tesla, Shopify, and Aviva have also experienced attacks, leading to substantial losses [8]. These incidents point toward the potential for zero-day attacks [9] and the necessity for frequent vulnerability assessments [10–12] and attacks detection [13–25].

Safeguarding containerized environments against security challenges is paramount. Containers, sharing the same physical system but not necessarily belonging to the same tenant, create a unique security landscape. The responsibility of ensuring the isolation, integrity, and availability of these environments falls on the service providers. Identifying intrusive behaviors in cloud containers and containerized applications is therefore paramount.

The architectural model of container technology introduces four main types of attacks: attacks from applications within the container, from the host to the container, between containers, and from a container to its host. While existing cryptographic technologies are crucial for cloud computing security, they come with significant computational overhead [26]. An Intrusion Detection System (IDS) is essential for securing cloud infrastructure and maintaining trust in virtualization [27, 28]. However, traditional Virtual Machine (VM)-based IDS models, due to their complexity, are less effective in containerized environments [6].

The rapidly evolving nature of cyber threats, especially in containerized cloud environments, necessitates the development of robust security solutions. Conventional IDS methods have limitations in accuracy, resource efficiency, and adaptability. This situation poses a critical challenge: creating an IDS that accurately identifies threats with minimal false-positives/negatives and functions efficiently in dynamic, resource-limited settings. Our proposed DL-HIDS model is developed to meet this challenge, aiming to enhance cloud environment security effectively.

Some researchers have used traditional, comparison-based methods for analyzing system call sequences or frequencies, which can be resource-intensive due to the need for constant comparisons [13, 14, 17]. Also, many have shifted to Machine Learning (ML) techniques, integrating them with system call features like Bag-of-System-Calls (BoSC) and N-grams [16, 18]. ML-based approaches identify patterns and predict based on new system call sets. Some authors propose using system calls as input for DL models, training them on these features for later detection [21–23]. However, these models often lose crucial system call elements, and while some preserve process information, they fall short in feature selection and other aspects. Consequently, existing IDS methods struggle with false-positives/negatives, efficient IDS placement, storage management, high impact, time consumption, frequent relearning, and effective feature selection [13–24].

Considering these difficulties, developing an advanced IDS that can effectively address and neutralize threats in containerized cloud environments becomes an immediate priority. The proposed DL-HIDS model addresses the need by leveraging DL techniques to analyze and interpret complex patterns in system data. Focusing on image features that preserve system call information, the model uses CNNs, renowned for their effectiveness in image analysis tasks such as recognition, detection, and segmentation. The DL-HIDS model is designed to adapt to the unique

challenges of containerized infrastructures, offering enhanced protection against a broad spectrum of malicious activities and evolving threats in the cloud. Thus, the following are key contributions of our proposed DL-HIDS for the containerized environment:

- In the proposed DL-HIDS model, the system call logs and their respective meta-data are utilized for feature engineering. The extracted features are transformed into grayscale images of various sizes using 2D vector after feature selection. This provides a novel approach to process and analyze system call data.
- The DL-HIDS presents an automated IDS model employing pre-trained deep CNN features that aim at improving attack classification within containerized cloud environments. The proposed model is experimented with two and three convolution layers consisting of deep features. Experiments were also carried out with various combinations of activation functions across different layers.
- The proposed model is validated using the LIDDS-2019 dataset which encompasses vulnerabilities leading to attacks in containerized cloud environments. This dataset provided a comprehensive foundation for assessing the effectiveness and robustness of our model in a real-world context.
- The DL-HIDS introduces an approach to intrusion detection, selecting features based on correlated system call attributes and metadata to create grayscale images for detecting intrusions. This shows significant improvement as compared to existing models with a specific focus on enhancing detection capabilities in containerized cloud environments.

The paper is organized as follows: Section 2 discusses the threat model and attacks on containerized system calls of the cloud. Section 3 elaborates on related work. Section 4 addresses the proposed system model with all its components. Section 5 presents a comparative evaluation of our system model, including results and discussion. Section 6 concludes our work with some future directions.

2 Vulnerabilities and threat model

Before submission to the repository, container images undergo multiple security measures. Despite the existence of security checks, there are various entry points that an attacker can exploit to gain access to the system. Figure 2 depicts the point of entry for an attacker into a containerized cloud environment. The developer may inadvertently use sensitive commands to build the image transforming it into a vulnerable one. As a result, the attacker may maliciously use these same sensitive commands to create a malicious image. These images are uploaded to the repository for public use. When a user downloads these malicious or vulnerable images from the repository, an attacker may exploit the vulnerability or malicious image to launch an attack against the user.

The emergence of a threat is possible as a result of several vulnerabilities [10] caused by the following factors:

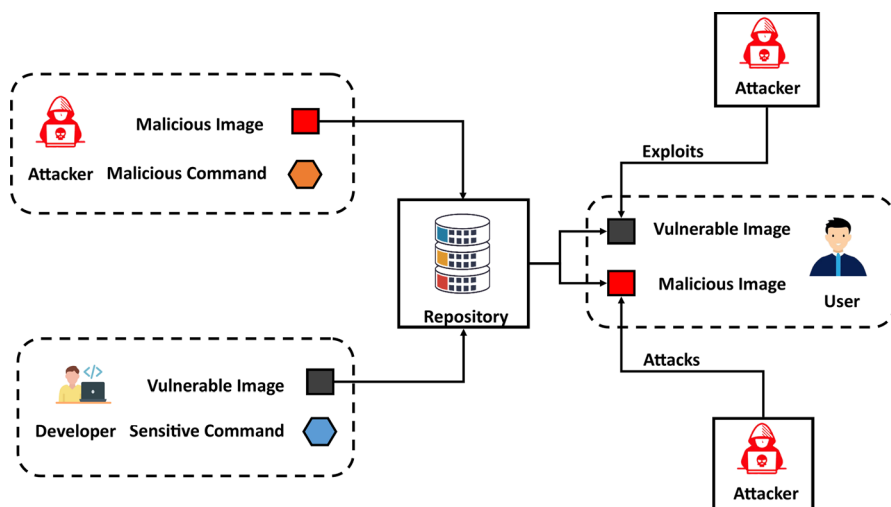


Fig. 2 Endpoints for the attacker into containerized cloud environment

1. **Vulnerable application code:** The software code or its third-party dependencies are integrated into an image and can create a security risk to a system.
2. **System crash vulnerability:** The vulnerability in the systemd package can crash the entire system if a local attacker can mount a filesystem on a very long path. This can impact containers running on the same host system.
3. **Badly configured container images:** Container images configured with a lack of foresight may exhibit vulnerabilities throughout the later phases of their life cycles. Privileges should be avoided to be granted to such types of images during configuration.
4. **Build machine attacks:** If an unauthorized actor manages to modify the building process of an image, it becomes possible for them to insert malicious code that can circumvent the production environment.
5. **Supply chain attacks:** A malicious actor can inject specific code into container images hosted in widely recognized and trusted image repositories, which can be downloaded by anyone without prior knowledge of the contents within.
6. **Badly configured containers:** This results from running a container with unexpected privileges with an unknown configuration file.
7. **Vulnerable hosts:** The host machine on which the container is deployed may be vulnerable, which can result in sensitive data thefts.
8. **Exposed secrets:** In the containerized cloud environment, tokens, credentials, and passwords used in an application could be exposed if not securely transferred between the destination and source without violating the security of other components.
9. **Container escape vulnerabilities:** This type of threat can occur at the weak points of the container runtime by escaping from its defined locations in the environment.

Vulnerabilities within a system can arise from the actions of three categories of entities referred to as honest, semi-honest, and malicious adversaries. Honest adversaries are those who adhere to the prescribed protocol and execute it according to established rules. Semi-honest adversaries, on the other hand, may passively collect information without deviating from acceptable protocols. Malicious adversaries actively engage in disruptive behavior, such as collecting information, cheating, disrupting the system, or causing harm to other system components, thereby deviating from the established protocols.

2.1 Threat model

The threat model for the containerized cloud environment is depicted in Fig. 3. The host application, Host OS, Container, container image from repositories, and the application within the container may act as semi-honest or malicious entities, thereby creating vulnerabilities. These vulnerabilities can be classified into the following categories of attacks:

1. Container to Container Attack
2. Application to Container Attack
3. Host to Container Attack
4. Container to Host Attack

All these four attacks are likely to occur in the containerized cloud environment. Our proposed model is capable of addressing these attacks.

2.2 Attack taxonomy

Due to the multiple vulnerabilities present in containerized cloud environments, there are several potential attack vectors that can be exploited. The attacks that can be launched on containerized cloud environments are outlined as follows:

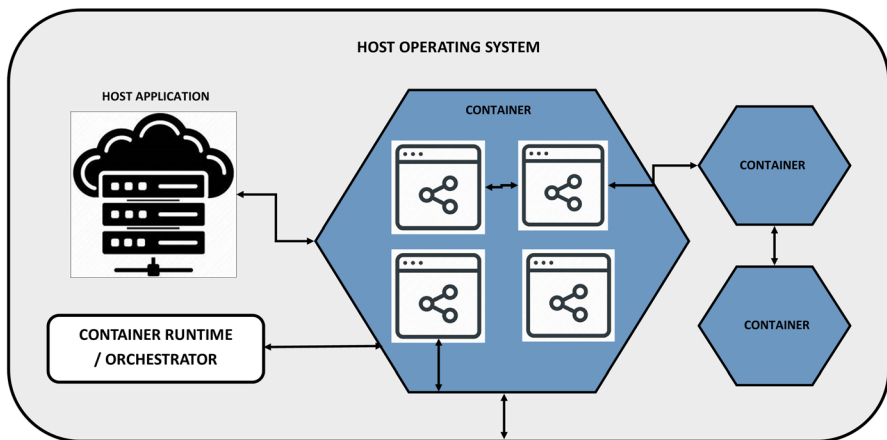


Fig. 3 Threat model for containerized cloud environment

1. **Container runtime escape attack:** An attacker attempts to get root privileges by escaping from the container to the host. If an attacker is able to obtain privileges, he or she will have complete access to all resources [29].
2. **Unauthorized access:** The attacker controls kernel root capabilities through memory and file modifications [30]. Using altered root credentials, the attacker can launch malicious programs with modified file characteristics or rewrite memory data to obtain privileges. Unauthorized access can inflict harm in a variety of ways, including by targeting containers and the host system. Therefore, illegal access of any kind should be prevented [1].
3. **Masquerading of user and resources:** By changing their resources, attackers may attempt to appear authentic or benign to users or security monitoring tools. While naming or placing the location of container resources, an attacker may reproduce or imitate the name or the location [10]. For instance, while retrieving the container image from the repository, the image having several potential attack vectors may be downloaded with name similar to legitimate image, can introduce security threats.
4. **Backdoor:** Untrusted container images may have a backdoor already installed. When these images are retrieved from the repository, the security mechanism is bypassed, and they are executed in the container. A container with a backdoor could allow an attacker to get root rights or execute malicious code in the future [1].
5. **Network-based intrusion:** When the container is poorly configured, attacks such as access to analyze all communication, including all unicast traffic and address resolution protocol spoofing, are possible. That may result in illicit conduct and data leakage [30].
6. **Shared libraries exploit:** The container daemon loads shared library files with root privileges, which might lead to privilege escalation [31].
7. **Denial of Service (DoS) attack:** The attacker can cause the application to crash by sending a significant number of requests [32]. The sharing of kernel resources by all containers in a container-based architecture makes application crashing tasks easy.
8. **Cryptomining attack/crypto-hijacking:** During application deployment and running of the container application, hidden malware, such as a crypt mining process, may run in the background and eventually hijack the container to access its computer resources and [6]. Using the CPU, GPU, and RAM of the victim, an attacker can exploit this vulnerability to mine cryptocurrency.
9. **Man-In-The-Middle attack:** An additional malicious component is injected between the data transmitter and the data receiver. People, containers, services, and middleware can all be spoofed [6].

IDS plays a crucial role in identifying and responding to potential threats in real-time. They analyze system activity to detect and alert security teams to suspicious behavior or possible attacks, helping to protect against cyber threats and mitigate potential damage. The containerized cloud environment can be protected with the help of IDS. The following are the mathematical proofs for the IDS in the containerized environment of the cloud:

- **Lemma 1:** Let S be the security of a containerized cloud environment and I be the isolation of containers. Then, the security of a containerized cloud environment is a function of the isolation of containers, such that: $S = f(I)$, where f is a function that maps the isolation of containers to the security of the containerized cloud environment.

Proof: The function $f : I \rightarrow S$ describes how container isolation impacts the overall security of the cloud environment. Here, I refers to various levels of container isolation, influenced by the containerization technology's isolation capabilities, security features implemented by the cloud provider, and user practices. S represents the resulting security levels. Thus, f captures the transformation from isolation levels to corresponding security states.

- **Lemma 2:** The effectiveness of an IDS in a containerized environment, denoted as E , can be mathematically represented as $E = f(M)$, where M symbolizes the system's ability to monitor container behavior.

Proof: The function $f : M \rightarrow E$ illustrates the relationship between monitoring capability M and the effectiveness E of the IDS. The domain M is characterized by factors like resource utilization (R), network traffic (N), and system events (S), formulated as $M = g(R, N, S)$. The co-domain E signifies the IDS's effectiveness. The function f thereby models how the monitoring capability influences IDS effectiveness.

- **Lemma 3:** The accuracy A of an IDS is defined as the proportion of correctly predicted attacks to the total number of actual attacks. This can be mathematically represented as $A = f(A')$, where A' denotes the usage of appropriate algorithms for data analysis.

Proof: The function $f : A' \rightarrow A$ demonstrates how the use of algorithms affects IDS accuracy. Here, $A' = g(T, C, Q)$, with T , C , and Q representing the type, complexity, and quality of algorithms, respectively. The domain A' encompasses these algorithmic aspects, while the co-domain A is the resultant accuracy of the IDS. This formulation emphasizes the importance of algorithm selection, complexity, and data quality in determining IDS accuracy.

Based on these lemmas, It can be concluded that the security of a containerized cloud environment can be improved by implementing an IDS that provides the ability to monitor the behavior of containers, uses appropriate algorithms for data analysis, and is designed to take into account the isolation provided by containers.

3 Related work

IDS is a device or software application or combination of hardware and software that monitors a network or system for malicious activities or policy violations. Figure 4 shows the classifications of the IDS for the containerized cloud environment which is based on the approach of the model selected by the researcher.

Several IDS techniques for the containerized cloud environment are in detail mentioned in Table 1.

In the landscape of containerized environment security, various IDS have been developed, ranging from traditional comparison-based methods to ML approaches. Abed et al. [13] focused on anomaly detection for Linux containers using the BoSC method without performance evaluation. Azab et al. [14] extended this to real-time environments, creating a database for recording simulation behaviors for various attacks but encountering high FPR. Liang et al. [15] identified a critical oversight in prior systems regarding the semantics of system actions and proposed a more feature-rich approach, though without conclusive evaluation. Gantikow et al. [19] used a rule-based approach with tools like Sysdig and Falco for monitoring and policy definition, effective in detecting misuses but potentially burdensome on the processor. Kashkoush et al. [20] introduced the Resilient IDS (RIDS) that relies on N-gram counts for anomaly detection, which might be impractical for real-time operations due to feature overload.

ML-based IDS, such as those proposed by Aljebreen et al. [16], Srinivasan et al. [17], and Li et al. [18], have also been explored. Aljebreen et al. combined BoSC with a sliding window in an ML framework but faced high false-positives. Srinivasan et al. addressed the loss of critical information in BoSC-only approaches by

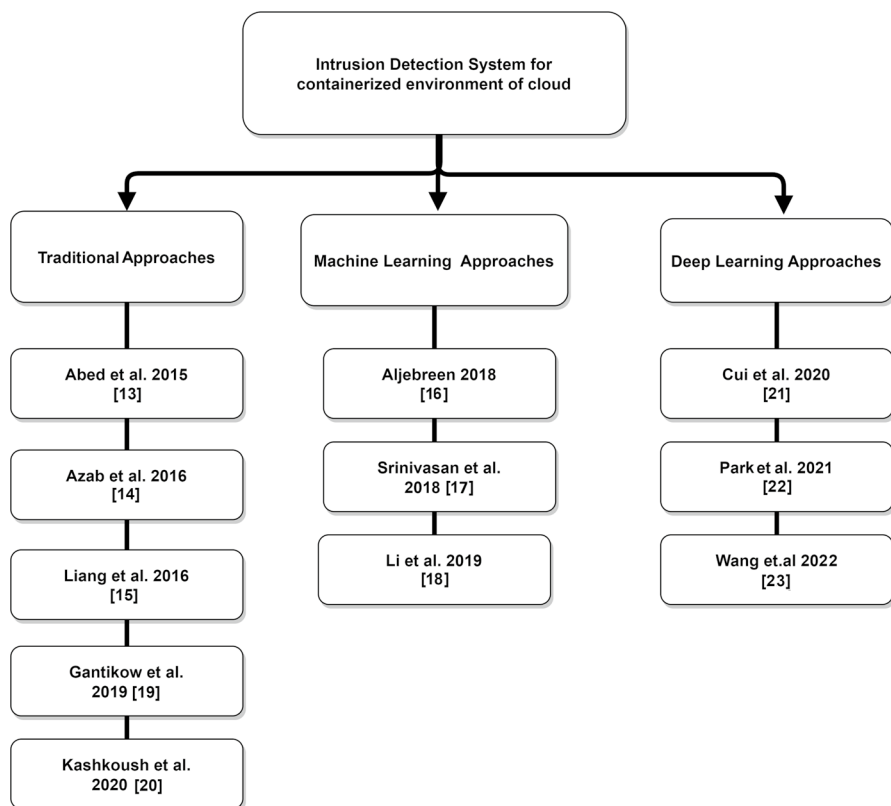


Fig. 4 Classification of IDS for containerized cloud environment

Table 1 IDS on the containerized environment of cloud

Paper	Features	Characteristic(s)	Remark(s)
[13]	BoSC	Protection against the disclosure of working on the application.	Work is done in a mode called 'offline.' Rates of false-positives are pretty high. Unauthorized users can easily bypass it.
[14]	BoSC	Working in an online mode and protection from the disclosure of working on the application.	Rates of false-positives are comparatively high.
[15]	CP relationship	Inter-process communication between systems is utilized and gives good results in terms of attack detection.	Only the number of features is examined. Two-step detection results in a relatively complicated approach.
[16]	BoSC	Combination of Frequency Plus BoSC with the ML algorithms.	FPR is comparatively high.
[17]	N-grams	Scalability in terms of running several containers on a single monitoring IDS.	Low performance, inaccurate in terms of false-positive and false-negative.
[18]	Raw system calls	Ensemble method for KNN and K-means for the learning of data gives efficient results.	N-grams has many duplicate entries which may affect the performance.
[19]	Raw system calls	Rule-based approach to check various constraints.	High impact on the process when the processed dataset is large.
[20]	BoSC	Protection from the disclosure of working on the application.	FPR are comparatively high if the ML approach is utilized.

Table 1 (continued)

Paper	Features	Characteristic(s)	Remark(s)
[21]	Raw system calls	Use of the neural network auto-encoder gives excellent results.	Time-consuming in terms of feature extraction.
[22]	Vector-to-Image	Lower the training costs compared to other methods, i.e., it takes less time in training.	The results are too low in terms of accuracy and other parameters.
[23]	Raw system calls	Uses variational auto-encoder to get better training and testing results.	Time consuming than the existing IDS.

using N-grams for behavior comparison, albeit increasing the feature count and comparisons needed. Li et al. utilized supervised and unsupervised models for anomaly labeling with N-grams, achieving high accuracy with random forest algorithms after data pre-processing.

Cui et al. [21] explored unsupervised introspection in multi-tenant applications, focusing on application-level security but found the process time-consuming. Park et al. [22] proposed pre-processing system call vectors into grayscale images for feature extraction, using Siamese Neural Networks (SNN) for anomaly detection. While this method reduced training costs, its overall effectiveness was lower than other approaches.

Studies were conducted on various domains to detect attacks using different DL approaches, as cited in [33–37]. These approaches utilize vector-to-image conversion of the features. In [33], Kim et al. proposed an LSTM-based, behavior-based host IDS and used the ensemble approach to address the high FPR. However, this approach has not yet detected packed malware. Upadhyay et al. [34] uses a CNN model for the IDS, and the KDD dataset was employed to evaluate the model. The authors selected 36 features out of the available ones and demonstrated that CNN outperforms SVM and Deep belief networks. The vector was converted into a 6x6 size, and the remaining function was performed on it. In [35], Khan et al. found that traditional ML approaches have a low detection rate, and feature engineering is a challenging task. To address this issue, CNN with the Softmax function was employed for IDS using the KDD dataset. Ni et al. [36] used CNN to classify malware by converting binaries into images. They generated the data themselves and used several techniques such as multi-hash, major block selection, and bilinear interpolation to enhance performance. Finally, Vinayakumar et al. [37] evaluated experiments with DNNs and other classical ML classifiers on various publicly available benchmark malware datasets. They used hyperparameter selection methods with the KDDCup 99 dataset to select the optimal network parameters and network topologies for DNNs. A new technique using deep reinforcement learning (DRL) is proposed for detecting and classifying dangerous cyber-attacks. The proposed deep

State-Action-Reward-State-Action (SARSA) model combines a SARSA-based reinforcement learning algorithm with a deep neural network for intrusion detection systems [38]. The performance of the method was validated using two benchmark datasets, NSL-KDD and UNSW-NB.

3.1 Research findings

Some approaches are promising but create performance overhead [15, 19], and some other approaches have high false-negatives and false-positives [13, 14, 16, 17, 22]. DL models have the disadvantage of having to re-learn each time the attack is added.

Also, Shojafar et al. [39] use the feature selection by optimizing the parameters of selection with various optimizing algorithms such as Artificial Bee Colony (ABC), Particle Swarm Optimization (PSO), and Differential Evolution (DE). The manuscript particularly signifies the importance of selecting the important parameters for detection.

The time required to re-learn is higher than that of the ML model [21, 22]. The existing work [22] with vector-to-image conversion has a limitation. The DL models run twice, and then the distance in results is calculated to find the attack. For vector-to-image features, the image size can be further improved so that more features can be incorporated into it. Furthermore, the generation of 2D vectors or images entails the utilization of raw metadata, which is then subjected to a process of feature selection to produce a grayscale image.

4 DL-HIDS: Deep Learning-based Host Intrusion Detection System

The proposed model's architecture for intrusion detection is shown in Fig. 5. Each module provides a more in-depth description of the specific detection component that the proposed architecture utilizes.

DL-HIDS is specifically designed for the detection of attacks in a containerized cloud environment including the attack missed during sequential and frequency-based detection. DL-HIDS implementation is easily adaptable for all scenarios of attack detection. The data from the behavior of containers are gathered and stored on the host system. It is assumed that the data collected are secured and from a trusted domain that cannot be breached.

System calls are the way for programs to interact with the OS. Therefore, the behavior of the system is known through the sequence of activities done by the system. Observation of the normal behavior of a system includes monitoring the sequence of system calls made while the system is operating without any vulnerabilities or attacks. This process also encompasses hardware-related and software services such as accessing a hard disc drive or device camera, as well as the creation of new processes and execution of new processes and communication with core kernel functions such as process scheduling. System calls serve as an essential communication link between a process and the operating system. Not only the sequence of system calls is examined, but also their process ID and system call arguments. Our

proposed system does not necessitate any prior knowledge about the system or the applications running on the system.

4.1 Data collection phase

The IDS is placed on the host kernel to monitor system calls between the container and the host kernel. There exist several tools to collect the system calls such as Sysdig [40], ptrace, strace, and Perf [41]. Strace and ptrace are in-built into Linux OS. Sysdig is used by authors [13, 14, 16–21] to collect system call for monitoring the containers. The reason behind choosing the Perf tool is comprehensively analyzed and clearly outlined in Table 2.

Perf tool [41] collects system call data from the containers that are running on the host kernel. Here, containers with different applications are hosted on the host OS with the Runtime Container Engine to manage the container. The setup is done for both Docker containers and Linux containers. Further, the system calls are collected for behavior analysis. These system calls are the logs of the system utilization in the form of CPU utilization, memory utilization, user interaction, and interaction with system hardware or devices. The following are the problems in the collection of data in a real-time environment:

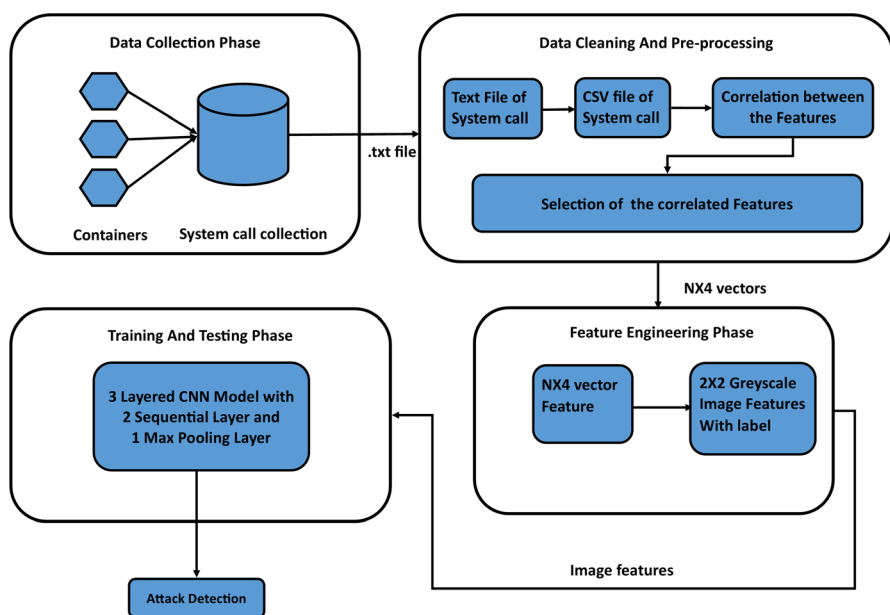


Fig. 5 Proposed IDS architecture for the containerized environment of cloud

- **Performance overhead:** The overuse of the system call collection tool may sometimes cause performance overhead and slow down the process execution which results in a reduction in the system call capture.
- **Security measures:** If the access credential gets compromised, system may not collect system calls and not be able to generate an alarm. Thus, there can be a mechanism in the IDS that alerts if the data do not get collected during several intervals of time.
- **Storage overhead:** The number of system calls generated is high, and it may cause overhead to the storage. SPEAKER [42] is an approach to split the collection of the data during the run time and the boot time. To save the system from storage overhead problems, one may use SPEAKER and take the data during the runtime, and the data can be dumped after the training of the system and detection is done once.

Algorithm 1 Data Collection from Log

Require: LF	▷ Log Files
Ensure: $FinalTraceLog$	▷ Traces file

```

1: function FETCHFILE( $dir$ )
2:   Initialize an empty list:  $logFiles \leftarrow []$ 
3:   for each file in specified directory do
4:     if file has ".txt" extension then
5:       Construct file path:  $filePath \leftarrow \text{concatenate}(\text{directory}, \text{file})$ 
6:       Try to open  $filePath$  and read its content
7:       if no error then
8:         Append content to  $logFiles$ 
9:       else
10:        Handle file reading error
11:      end if
12:    end if
13:  end for
14:  return  $logFiles$ 
15: end function
16: function DATACOLLECTION( $LF$ )
17:    $FinalTraceLog \leftarrow \emptyset$ 
18:    $F1 \leftarrow \text{FETCHFILE}(dir)$ 
19:   for all Trace in  $F1$  do
20:      $TraceLog \leftarrow \text{take } n\text{-second log from Trace}$ 
21:      $FinalTraceLog \leftarrow \text{append}(TraceLog)$ 
22:   end for
23:   return  $FinalTraceLog$ 
24: end function

```

Table 2 Comparison between Perf and Sysdig

Aspect	Perf	Sysdig
Integration	Integrated directly with the Linux kernel, offering access to kernel-level performance counters.	Operates primarily in user space, with kernel module for deep system inspection.
Performance Overhead	Generally lower, especially in high-frequency event monitoring due to kernel-level integration.	Can be higher due to broader system activity monitoring and user space operation.
Data Collection	More focused on specific performance metrics and system calls.	Provides a broad overview of system activities, including file I/O, network, and process interactions.
Flexibility	Highly flexible in event selection, allowing for tailored data collection specific to research requirements.	Broad but less customizable in terms of event selection compared to Perf.
User-Friendliness	May require more expertise for setup and configuration due to kernel-level operations.	Generally more user-friendly with easier setup and more intuitive interfaces for broader system monitoring.
Resource Efficiency	More efficient in resource-constrained environments due to focused data collection and lower overhead.	Less efficient in such environments due to broader data collection scope.
Real-Time Analysis	Highly capable in real-time performance analysis due to direct kernel access.	Capable, but potentially less efficient for real-time analysis in high-performance or resource-limited settings.

Algorithm 1 for collecting system call data via Perf is outlined, highlighting the process of forming trace files from the Perf-tool data. These trace files are then parsed during the Pre-processing Phase, with the complexity of the Data Collection algorithm being $O(n)$. This methodology ensures efficient, accurate, and low-impact data collection, crucial for the effectiveness of our DL-HIDS model in a containerized cloud environment.

4.2 Data cleaning and pre-processing phase

Data pre-processing is a critical initial stage in developing a machine learning model, where structured data are prepared and refined. This process begins with converting system call trace files, typically in text format, into a more structured CSV format using Python scripts. Subsequently, the data undergo cleaning and are assigned numeric semantics using the MultiLabelEncoder. This tool is particularly adept at handling multiple categorical variables, streamlining the preprocessing phase, and ensuring compatibility with scikit-learn pipelines, a common framework in machine learning applications.

The MultiLabelEncoder, an extension of scikit-learn's LabelEncoder, is adept at encoding multiple columns simultaneously, a feature that significantly enhances its utility. It assumes that all specified columns, or all columns by default in a DataFrame, are categorical. However, a notable consideration with this approach is the potential introduction of an artificial ordinal relationship among categories, as it translates categorical data into numerical values. Such a transformation may not be suitable for models that interpret numerical features as algebraic quantities, a factor that needs consideration depending on the specific model and context.

In the data pre-processing workflow, vectors are generated from the encoded data, which include the system call and its arguments as determined by the process. This workflow, encompassing data cleaning, pre-processing, and feature selection, is illustrated in Fig. 6. The first two steps depicted in the diagram correspond to data cleaning and pre-processing, setting the stage for effective feature selection.

4.3 Feature engineering phase

Upon analysis of the CSV files after pre-processing, it was observed that the fields 'A', 'C', 'D', 'E', and 'F' exhibit certain characteristics. Field 'A' represents a numeric identifier for each log line, while fields 'C', 'D', 'E', and 'F' contain redundant values across the dataset. Consequently, these fields are likely not critical in influencing the outcome of the analysis. However, to avoid relying solely on assumptions, a correlation matrix was computed to assess the relationship of these fields with the provided labels. The results indicated that these fields have either a negative or low correlation with the label. Therefore, the focus shifted to fields with a correlation higher than 0.4. This criterion led to the selection of fields 'B', 'G',

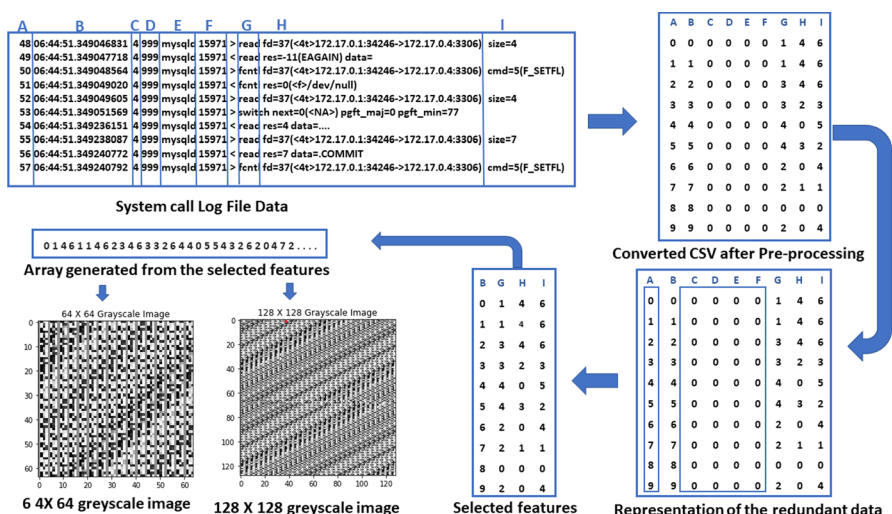


Fig. 6 Log files converted to grayscale image features

'H', and 'I', which demonstrated correlations of 0.52, 0.47, 0.60, and 0.62, respectively, indicating their higher relevance in the analysis.

Continuing from the feature selection phase, the identified four features from the system call data are organized into an $N \times 4$ matrix, where N represents the total number of system calls captured. To facilitate further analysis, this matrix is flattened into a one-dimensional array. This transformation is crucial for image creation, where each sample is mapped to a value within the 0–255 range that supports standard image pixel value conventions. The conversion to this format is efficiently handled by a Python script that ensures that each system call feature is appropriately represented in the one-dimensional array for subsequent processing stages.

Subsequently, the one-dimensional array was restructured into a two-dimensional $N \times N$ array to form a 2D image, adhering to a single-color channel grayscale format ($M \times N \times 1$). For example, for a target image size of 128 pixels, the array was segmented into 16384 elements (128×128), each representing a pixel in the grayscale image. This step is critical for converting system call data into a format suitable for image-based methods.

Algorithm 2 outlines the conversion of system calls to an image. Each vector from the pre-processed data is appended to the image data, which is then transformed into an image and stored in a specified folder. The main function `VECTOR_TO_IMAGE()` with a complexity of $O(n)$ efficiently handles this transformation, facilitating the transition from system call vectors to image data for analysis.

Algorithm 2 System calls-to-image conversion

Require: *vectorData* ▷ vectors of the Data
Ensure: *Path* ▷ Path to the grey scale Image

```

1: function CONVERTIMAGE(imageData)
2:   Perform grayscale conversion on imageData
3:   Save the grayscale image to a file
4:   return Path of the saved image file
5: end function
6: function VECTOR_TO_IMAGE(vectorData)
7:   for all vector in vectorData do
8:     if sizeOfData  $\neq N^2$  then
9:       imageData  $\leftarrow$  vector.append()
10:    end if
11:    greyscaleImg  $\leftarrow$  CONVERTIMAGE(imageData)
12:    store greyscaleImg file(Path)
13:  end for
14:  return Path
15: end function

```

4.4 Training and testing phase

CNN is used as a powerful classification model for intrusion detection. The basic structure of a CNN for image classification consists of multiple layers, including convolution, activation, pooling, and fully connected layers, as depicted in Fig. 7.

The core mathematical building block of a CNN is the convolution operation, which involves sliding a filter over the input image to compute a dot product at each position as shown in Equation 1.

$$y(i, j) = (x * f)(i, j) = \sum_{k=1}^K \sum_{l=1}^L x(i+k-1, j+l-1) * f(k, l) \quad (1)$$

where $y(i, j)$ is the output feature map, x is the input image, f is the filter or kernel, and K and L are the dimensions of the filter. The convolution operation slides the filter over the input image and computes the dot product at each position.

In this article, the CNN used for intrusion detection has 2–3 convolution layers and 1–2 hidden classification layers, allowing it to be sensitive to the various aspects of the 2D feature vector, the image. The use of pooling and the ReLU activation function contributes to an appropriate model, as it reduces computation cost and results in binary outputs, speeding up training and computing time. These unique qualities make CNNs exceptional image classification tools that have demonstrated state-of-the-art performance on various benchmark datasets.

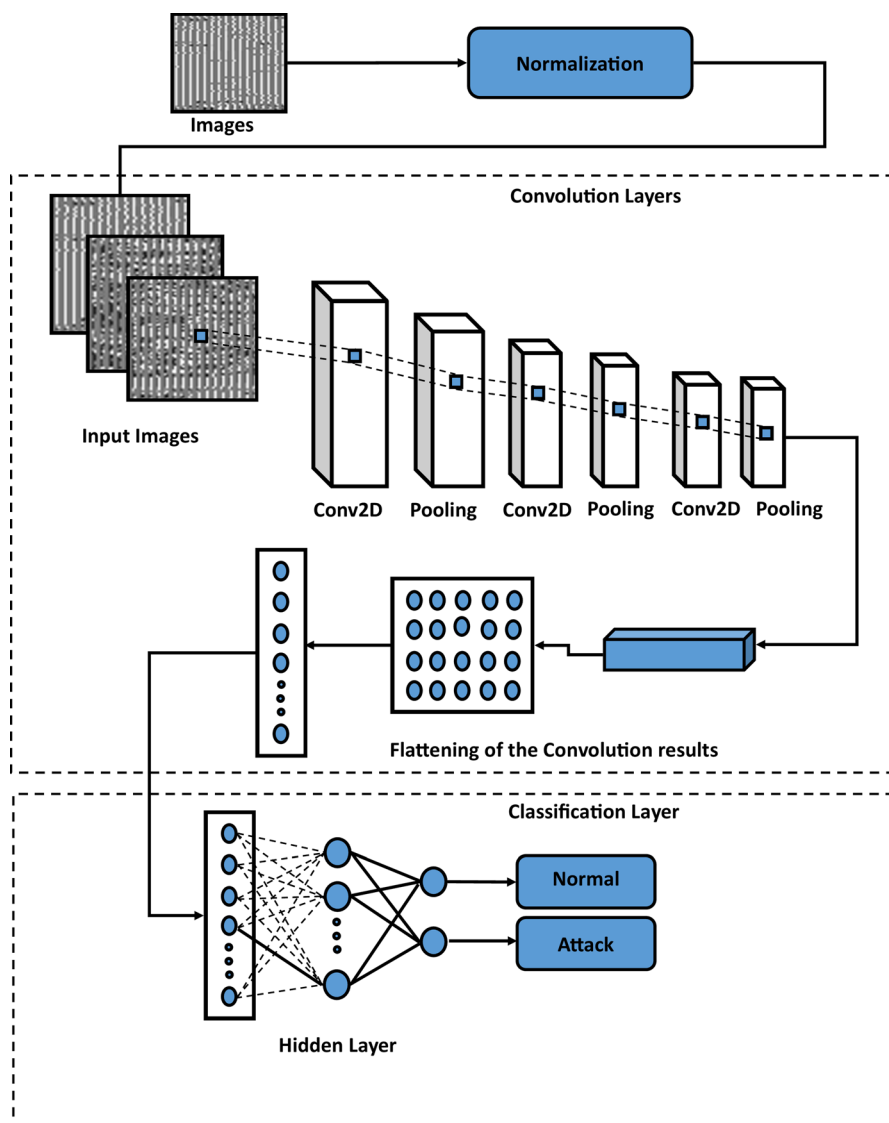


Fig. 7 CNN diagram with its layers

5 Performance analysis and comparisons

5.1 Dataset

For validation, well-known HIDS datasets have been considered. Table 3 provides the comparison among the available datasets.

Here, the parameters provided by the datasets are discussed. The containerized cloud environment is used for collecting the data by the LIDDS. Therefore, LIDDS is selected to evaluate work.

5.1.1 Description of LIDDS dataset

The LDDIS is developed by taking data from containers that reside on the host operating system's kernel. There is a very clear description of the system's normal and abnormal behavior call traces. The dataset incorporates many different vulnerabilities and attacks namely BruteForce login (CWE-307), Authorization bypass (CVE-2012-2122), Heartbleed (CVE-2014-0160), Integer overflow (CVE-2017-7529), Information Leak (CVE-2018-3760), Conflict of interest of Content (CVE-2019-5418), File Upload (CWE-343), SQL injection (CWE-89), and Zip Slip (CWE-29). These are among the top vulnerabilities found in Docker [10].

5.2 Performance metrics

Accuracy, Precision, Recall, F1 score, and False-Positive Rate (FPR) are well-defined measures used to evaluate the attack detection ability of ML models [47].

Accuracy is the fundamental parameter used to evaluate the performance of classification models. It is the proportion of right identifications relative to the total number of classifications.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

Precision is often defined as the classifier's ability to avoid identifying a negative sample as positive. The ratio is

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

Recall is a classifier's ability to identify all positive samples. The ratio is

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

F1 score is the harmonic mean of Precision and Recall and gives a much better measure of the model.

$$F1Score = 2 * \frac{(P * R)}{(P + R)} \quad (5)$$

FPR is the percentage of negative cases in the data that were mistakenly reported as positive.

Table 3 Comparison of HIDS dataset

Dataset	Year	Methods of data generation	A	B	C	D	E	F	G	H
UNM [43]	1997	Unix with different command	✗	✗	✗	✓	✗	✓	✗	✗
KDD [44]	2009	Solaris with networks attack collected with basic security module	✓	✓	✓	✓	✓	✓	✗	✗
ADFA-LD [45]	2014	Ubuntu Linux and Metasploit	✗	✗	✗	✗	✗	✗	✓	✗
NGIDS [46]	2017	Network packets and host logs from PerfectStrom hardware	✗	✗	✓	✓	✗	✓	✓	✗
LIDDS [24]	2019	Containers environment and Metasploit	✓	✓	✓	✓	✓	✓	✓	✓

A = Arguments, B = Return values, C = Timestamps, D = Process ID, E = Metadata,

F = Threads, G = Data buffers, H = containerized environment

$$FPR = \frac{FP}{FP + TN} \quad (6)$$

where, TP is True-Positive, TN is True-Negative, FP is False-Positive, FN is False-Negative, R is Recall, and P is Precision.

5.3 Simulation results and comparison

The proposed approach was implemented using the Python-based Keras and Sklearn library in a Jupyter notebook. The experiments were conducted on a platform with a configuration of AMD Ryzen 5 3500, 6-core Processor 3.60 GHz, 32 GB RAM with a single NVIDIA GeForce GTX 1660 Super for the pre-processing, feature engineering, and training testing phase.

The files of the LIDDS dataset are pre-processed as mentioned in Sect. 4.2. The number of generated images is as per Table 4. The file of each attack is pre-processed and converted into 64x64, 128x128, 256x256, 512x512, and 1024x1024.

Experiments were conducted to compare various well-known image classification algorithms, such as the Support Vector Classifier, Gaussian Naive Bayes Classifier, Decision Tree Classifier, Multi-Layer Perceptron K-Neighbor Classifier, and Proposed CNN Classifier. The images used in the experiment were 64x64 pixels in size and were trained for 15 epochs with Leaky ReLU and Sigmoid. Results of this comparison are presented in Table 5. It can be seen that the CNN classifier has the highest values for accuracy, precision, and F1 score, making it the best classifier in

comparison with others because of its properties such as Local Receptive Fields, Spatial Invariance, Parameter Sharing, Downsampling, and Transfer Learning, and it lags behind SVC for recall rate.

The DL-HIDS model experiments involved the careful selection of Leaky ReLU, Sigmoid, and ReLU as activation functions, each chosen for their distinct benefits in DL models designed for detecting security threats in containerized cloud environments. ReLU is favored for its simplicity and efficiency, particularly in addressing the vanishing gradient problem, making it suitable for CNNs where speed is key. To counter the 'dying ReLU' issue, Leaky ReLU was chosen for its ability to maintain a small gradient when inactive, thus ensuring continuous gradient flow in complex architectures - a critical feature for intricate models like ours. The Sigmoid function, pivotal in mapping inputs to a probability range, plays a crucial role in the final layer of our model for binary classification, enabling a probabilistic approach to threat detection. By integrating these functions in various combinations, we aim to leverage their distinct advantages to heighten the accuracy and reliability of our intrusion detection system within the cloud computing framework. Thus, a combination of the activation functions, i.e., (a) ReLU and Sigmoid, (b) ReLU and Softmax, (c) Leaky ReLU and Sigmoid, and (d) Leaky ReLU and Softmax, was experimented.

Figure 8 shows the results that are obtained on a total of three layers of convolution and one hidden layer with the above activation functions combination and at the epoch of 5 with a batch of 30 of the images during the training. The outcome of the experiments shows that the combination of Leaky ReLU and Sigmoid and ReLU and Sigmoid almost gives optimum results.

Parallel to our focus on activation functions was the deliberate choice of epoch counts 5, 10, 15, and 20 to examine the impact of training duration on the model's learning curve. Beginning with 5 and 10 epochs, our goal was to navigate the model's initial learning phase, where it is most susceptible to underfitting due to limited data exposure. These lower epoch counts set the baseline for the model's early learning stages. Increasing the count to 15 epochs transitions us into the optimal fitting zone where the model is hypothesized to have assimilated the data sufficiently to discern key patterns without the risk of overfitting. Pushing further to 20 epochs tests the limits of overfitting where the model might shift from generalizing to memorizing the training data. Monitoring the model's performance across these stages, especially looking for performance plateaus or declines in higher epochs, enables us to identify the most effective training duration. This meticulous approach is crucial for fine-tuning our DL-HIDS model, ensuring it is adept at learning efficiently and generalizing effectively for robust threat detection in containerized cloud environments.

The performance of the activation functions combinations at epochs 5, 10, 15, and 20 was evaluated, as shown in Table 6. The results indicate that the combination of Leaky ReLU and Sigmoid produced superior results. The evaluations were conducted using 64 x 64 sized images with each size trained for 5, 10, 15, and 20 epochs, and the results were measured using accuracy, precision, recall, F1 score, and FPR. Both types of CNNs perform well, with the Leaky ReLU and Sigmoid CNN generally performing slightly better.

Table 4 Feature image - LIDDS dataset

Attacks	Vulnerability	Type of files	No. of files	No. of images/size				
				64	128	256	512	1024
Brute-Force	CWE-307	Attack	98	4557	1138	284	71	17
		Normal	995	7316	1862	453	108	21
Authorization bypass	CVE-2012-2122	Attack	155	3243	810	210	46	11
		Normal	1241	5840	1461	319	77	16
Heart-bleed	CVE-2014-0160	Attack	100	710	177	44	10	2
		Normal	1000	6675	1664	412	99	20
Integer overflow	CVE-2017-7529	Attack	174	567	141	35	8	2
		Normal	983	3097	766	188	44	10
Information leak	CVE-2018-3760	Attack	137	5038	1277	314	77	19
		Normal	1084	33258	8477	1887	514	125
Conflict of interest of Content	CVE-2019-5418	Attack	98	3267	816	204	51	12
		Normal	981	32452	8162	2027	503	123
Unrestricted File Upload (PHP)	CWE-434	Attack	99	3267	816	204	51	12
		Normal	972	32452	8162	2027	503	123
Unrestricted File Upload (EPS)	CWE-434	Attack	98	23130	5769	1440	360	87
		Normal	968	235807	58909	14703	3625	907
SQL injection	CWE-89	Attack	100	12843	3201	810	198	47
		Normal	978	38786	9696	2419	602	147
Zip Slip	CWE-29	Attack	98	43426	10854	2710	669	162
		Normal	968	440621	110228	27555	32764	1701

Table 5 Comparison with other classifiers

Parameters	SVC	GNB	DT	MLP	KN	CNN
Accuracy	95	73	93	60	50	97
Precision	93	72	93	30	71	99
Recall	95	72	93	60	58	89
F1 score	93	73	92	45	42	93

The DL-HIDS model strategically employs image sizes of 64x64 and 128x128 pixels, designed to balance the level of detail in feature extraction

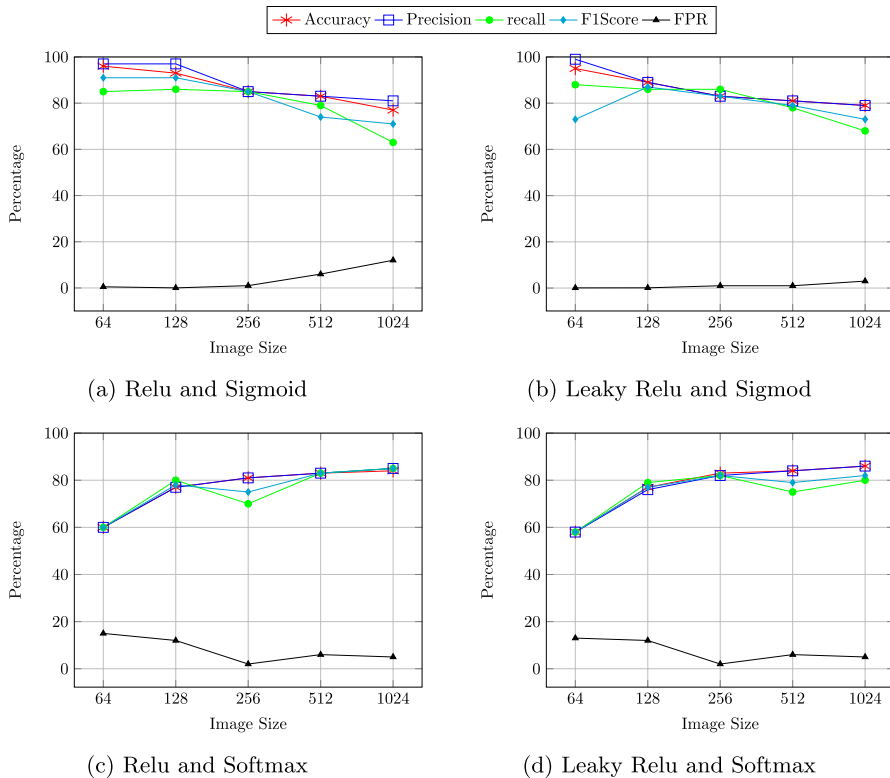


Fig. 8 Experimentation using different activation functions

with computational efficiency. The choice of a 64x64 pixel size was driven by the need for speed and efficiency in processing, which is especially important for rapid threat detection in real-time scenarios. This smaller size allows the model to quickly extract essential features without the overhead of processing large amounts of data, achieving a balance between capturing necessary details and maintaining computational simplicity. Conversely, the 128x128 pixel size was chosen for its ability to provide greater detail, enabling the model to identify finer and more complex features that might be overlooked in smaller images. Although this larger size requires more computational resources and lengthens training times, it is advantageous for detecting subtle and intricate threat patterns in containerized cloud environments.

The performance of the DL-HIDS model was further enhanced by expanding the CNN architecture to include both two-layered and three-layered configurations. These configurations were tested with the specified image sizes employing Leaky ReLU and Sigmoid activation functions which tended to yield better results. Experiments were also carried out using various training durations, notably epoch counts of 5, 10, 15, and 20. It was observed that models trained for 20 epochs typically achieved better outcomes than those with shorter training

Table 6 Results of (ReLU and Sigmoid) and (Leaky ReLU and Sigmoid) CNN on LIDDS dataset individual attack detection

Size	Epoch	Accuracy		Precision		Recall		F1 score		FPR	
		RS	LRS	RS	LRS	RS	LRS	RS	LRS	RS	LRS
64X64	5	96	95	97	99	85	85	91	91	0.5	0.75
	10	98	97	99	99	92	87	95	93	0.01	0.01
	15	98	97	99	99	91	89	95	93	0.02	0.05
	20	97	98	99	99	85	93	91	95	0.004	0.007

RS = ReLU and Sigmoid, LRS = Leaky ReLU and Sigmoid

periods. The three-layered CNN in combination with a four-feature vector showed superior performance for both 64x64 and 128x128 image sizes that are comprehensively compared and highlighted in bold in Table 7. It also displays the differences across configurations, image sizes, and training durations. This methodology was instrumental in evaluating the balance between processing speed, feature depth, and overall model effectiveness, aiming to optimize threat detection in containerized cloud environments.

The DL-HIDS model features comparison with Park's method [22], as elaborated in Table 8. A key difference between this model and Park's approach is evident in the feature selection process, where a detailed strategy was employed to eliminate redundant and non-correlated features before image creation. This refinement in feature selection was pivotal in our 64x64 architecture outperforming Park's model across all metrics. By filtering out irrelevant data, our model could more effectively concentrate on significant patterns, enhancing its ability to detect security threats.

Moreover, the choice of a 64x64 image size in our model demonstrated greater efficiency and effectiveness than the larger 128x128 size, as well as Park's approach. This smaller size, combined with our optimized feature set, facilitated a more focused and computationally efficient learning process. Our model's architecture by incorporating variations in layer depth and activation functions was specifically tailored to complement our selected features and image size. This fine-tuning helped in surpassing the performance of Park's method where the model might not have been as precisely adapted to handle the specific features and image dimensions. This comparison underscores the importance of targeted feature selection and architectural optimization in enhancing the performance of models designed for threat detection in containerized cloud environments.

DL-HIDS not only pushes the boundaries of current DL applications in cloud security but also opens new avenues for research, encouraging further exploration into the optimization of feature selection and model architecture for improved security solutions. From a practical point of view, the findings of our DL-HIDS model have significant implications for real-world applications in cloud security. The improved accuracy and efficiency in threat detection make the model highly applicable for businesses and organizations that rely on containerized cloud infrastructures. Its ability to identify potential threats can lead to quicker response times, minimizing

Table 7 Results of two-layered and three-layered CNN on LIDDS dataset individual attack detection

Size	Epoch	Accuracy		Precision		Recall		F1 score		FPR	
		2	3	2	3	2	3	2	3	2	3
64x64	5	93	95	90	98	83	85	86	91	0.16	0.75
	10	94	97	91	99	78	87	84	93	0.02	0.01
	15	94	97	91	99	79	89	84	93	0.03	0.005
	20	94	98	92	98	79	93	85	95	0.05	0.007
128x128	5	92	93	92	97	82	82	86	87	0.1	0.01
	10	92	93	92	98	82	83	87	90	0.07	0.04
	15	92	93	92	98	82	83	87	90	0.07	0.04
	20	92	93	93	98	82	84	86	90	0.07	0.04

2 = Two-layered CNN model, **3** = Three-layered CNN model

Table 8 Comparison between Park's approach and proposed CNN architectures (64x64 and 128x128)

Parameters	Park's [22]	64x64	128x128
Accuracy	93%	98.12%	93.58 %
Precision	93%	98.18%	98.48%
Recall	92%	93.60%	84.49%
F1 score	91%	95.82%	90.95%
FPR	1%	0.007%	0.04%

the risk of data breaches and system compromises. Additionally, the model's optimized computational requirements mean that it can be deployed in a wider range of cloud environments including those with limited resources. This adaptability enhances the accessibility and applicability of advanced security measures by making robust cloud security more attainable for various users. Overall, our research paves the way for more secure, efficient, and reliable cloud computing environments which is vital in an era where cloud technologies are increasingly integral to business operations and data.

6 Conclusion and future work

DL-HIDS marks a significant improvement in detecting malicious activities within containerized cloud environments. Our research demonstrates that the integration of the Perf tool and MultiLabelEncoder significantly reduces performance overhead, enhancing efficiency in information extraction from trace files. The novel vector-to-image approach transforms feature vectors into images and has proven to be a more effective method than traditional techniques, as evidenced by improved metrics such as accuracy, precision, recall, F1 score, and a lower FPR. The implementation of a three-layered neural network with Leaky ReLU and Sigmoid activation functions further strengthens these results. Despite our advancements, a notable

limitation is our model's reliance on the quality and diversity of the dataset, particularly LIDDS-2019. This underscores the necessity for ongoing data enrichment to maintain effectiveness against new threats. In future work, the emphasis will be placed on expanding the dataset that will improve efficiency in environments with limited resources and developing strategies for real-time adaptation. These steps are crucial for advancing DL-HIDS into a more robust, adaptable, and universally applicable solution for cloud security.

Funding The author did not receive any support from any organization for the submitted work.

Availability of data and materials Data generated during and/or analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of interest The authors declare that they have no known conflict of interest.

Consent to publish All authors have agreed to publication.

References

1. Sultan S, Ahmad I, Dimitriou T (2019) Container security: issues, challenges, and the road ahead. *IEEE Access* 7:52976–52996. <https://doi.org/10.1109/ACCESS.2019.2911732>
2. Moore S (2021) gartner prediction newsroom Sydney Australia. <https://www.gartner.com/en/newsroom/press-releases/2020-06-25-gartner-forecasts-strong-revenue-growth-for-global-co>. Accessed 05 Jul
3. bleepingcomputer news. <https://www.bleepingcomputer.com/news/security/over-900-000kubernetes-instances-found-exposed-online/>. Accessed 05 Jul 2021
4. Ahuje M (2020) New vulnerability exposes kubernetes to man-in-the-middle attacks. <https://www.tigera.io/blog/new-vulnerability-exposes-kubernetes-to-man-in-the-middle-attacks-heres-how-to-mitigate/>. Accessed 10 Dec
5. Prizmant D (2021) Siloscape: first known malware targeting windows containers to compromise cloud environments. <https://unit42.paloaltonetworks.com/siloscape/>. Accessed 10 Jun
6. Karn RR, Kudva P, Huang H, Suneja S, Elfadel IM (2021) Cryptomining detection in container clouds using system calls and explainable machine learning. *IEEE Trans Parall Distrib Syst* 32(3):674–691. <https://doi.org/10.1109/TPDS.2020.3029088>
7. Knowledge D.C (2021) aws-outage-that-broke. <https://www.datacenterknowledge.com/archives/2017/03/02>. Accessed 15 Dec
8. Gamage W.H (2021) common-container-security-threats. <https://www.wwt.com/article/common-container-security-threats>. Accessed 12 Dec
9. Maggi F, Matteucci M, Zanero S (2010) Detecting intrusions through system call sequence and argument analysis. *IEEE Trans Depend Secure Comput* 7(4):381–395. <https://doi.org/10.1109/TDSC.2008.69>
10. Common-container-security-threats. <https://www.cvedetails.com/vulnerability-list>. Accessed 15 Dec 2021
11. Martin A, Raponi S, Combe T, Di Pietro R (2018) Docker ecosystem—vulnerability analysis. *Comput Commun* 122:30–43. <https://doi.org/10.1016/j.comcom.2018.03.011>
12. Stopel D, Bernstein B (2021) Runtime detection of vulnerabilities in an application layer of software containers. Google Patents. US Patent 10,915,628
13. Abed A.S, Clancy T.C, Levy D.S (2015) Applying bag of system calls for anomalous behavior detection of applications in linux containers. In: 2015 IEEE Globecom Workshops (GC Wkshps), pp 1–5. <https://doi.org/10.1109/GLOCOMW.2015.7414047>

14. Azab M, Mokhtar B, Abed A.S, Eltoweissy M (2016) Toward smart moving target defense for linux container resiliency. In: 2016 IEEE 41st Conference on Local Computer Networks (LCN), pp 619–622. <https://doi.org/10.1109/LCN.2016.106>
15. Liang H, Hao Q, Li M, Zhang Y (2016) Semantics-based anomaly detection of processes in linux containers. In: 2016 International Conference on Identification, Information and Knowledge in the Internet of Things (IIKI), pp 60–63. <https://doi.org/10.1109/IIKI.2016.99>
16. Aljebreen M.J (2018) Towards intelligent intrusion detection systems for cloud computing. PhD thesis, Florida Institute of Technology, Melbourne, Florida
17. Srinivasan S, Kumar A, Mahajan M, Sitaram D, Gupta S (2019) Probabilistic real-time intrusion detection system for docker containers. In: Thampi, S.M., Madria, S., Wang, G., Rawat, D.B., Alcaraz Calero, J.M. (eds.) Security in Computing and Communications, pp 336–347. Springer, Singapore. https://doi.org/10.1007/978-981-13-5826-5_26
18. Li Z, Zhang Z, Liu X, Zhu C (2019) Anomaly detection for container cluster based on jointcloud platform. In: Proceedings of the 2019 3rd International Conference on Compute and Data Analysis. ICCDA 2019, pp 26–30. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3314545.3314567>
19. Gantikow H, Zöhner T, Reich C (2020) Container anomaly detection using neural networks analyzing system calls. In: 2020 28th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP), pp 408–412. <https://doi.org/10.1109/PDP50117.2020.00069>
20. Kashkoush M, Clancy C, Abed A, Azab M (2020) Resilient intrusion detection system for cloud containers. *Int J Commun Netw Distrib Syst* 24:1. <https://doi.org/10.1504/IJCNSD.2020.10025197>
21. Cui P, Umphress D (2020) Towards unsupervised introspection of containerized application. In: 2020 the 10th International Conference on Communication and Network Security. ICCNS 2020, pp 42–51. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3442530>
22. Park D, Kim S, Kwon H, Shin D, Shin D (2021) Host-based intrusion detection model using siamese network. *IEEE Access* 9:76614–76623. <https://doi.org/10.1109/ACCESS.2021.3082160>
23. Wang Y, Chen X, Wang Q, Yang R, Xin B (2022) Unsupervised anomaly detection for container cloud via bilstm-based variational auto-encoder. In: ICASSP 2022—2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp 3024–3028. <https://doi.org/10.1109/ICASSP43922.2022.9747341>
24. Röhling M.M, Grimmer M, Kreubel D, Hoffmann J, Franczyk B (2019) Standardized container virtualization approach for collecting host intrusion detection data. In: 2019 Federated Conference on Computer Science and Information Systems (FedCSIS), pp 459–463. <https://doi.org/10.15439/2019F212>
25. Jolak R, Rosenstatter T, Mohamad M, Strandberg K, Sangchoolie B, Nowdehi N, Scandariato R (2022) Conserve: a framework for the selection of techniques for monitoring containers security. *J Syst Softw* 186:111158. <https://doi.org/10.1016/j.jss.2021.111158>
26. Modi C, Patel D, Borisaniya B, Patel H, Patel A, Rajarajan M (2013) A survey of intrusion detection techniques in cloud. *J Netw Comput Appl* 36(1):42–57. <https://doi.org/10.1016/j.jnca.2012.05.003>
27. Srivastava N, Chaudhari A, Joraviya N, Gohil B.N, Ray S, Rao UP (2022) A review of machine learning-based intrusion detection systems on the cloud. In: Rao, U.P., Patel, S.J., Raj, P., Visconti, A. (eds.) Security, privacy and data analytics, pp 303–317. Springer, Singapore. https://doi.org/10.1007/978-981-16-9089-1_25
28. Chaudhari AR, Gohil BN, Rao UP (2023) A review on cloud security issues and solutions. *J Comput Security* 31(4):365–391. <https://doi.org/10.3233/JCS-210140>
29. Jian Z, Chen L (2017) A defense method against docker escape attack. In: Proceedings of the 2017 International Conference on Cryptography, Security and Privacy. ICCSP '17, pp 142–146. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3058060.3058085>
30. Souppaya M, Morello J, Scarfone K (2017) Application container security guide. National Institute of Standards and Technology. <https://doi.org/10.6028/nist.sp.800-190>
31. Barlev S, Basil Z, Kohanim S, Peleg R, Regev S, Shulman-Peleg A (2016) Secure yet usable: protecting servers and linux containers. *IBM J Res Develop* 60(4):12–11210. <https://doi.org/10.1147/JRD.2016.2574138>

32. Chelladhurai J, Chelliah P.R, Kumar S.A (2016) Securing docker containers from denial of service (dos) attacks. In: 2016 IEEE International Conference on Services Computing (SCC), pp 856–859. <https://doi.org/10.1109/SCC.2016.123>
33. Kim J, Kim J, Kim H, Shim M, Choi E (2020) Cnn-based network intrusion detection against denial-of-service attacks. *Electronics* 9(6):916. <https://doi.org/10.3390/electronics9060916>
34. Upadhyay R, Pantiukhin D (2017) Application of convolutional neural network to intrusion type recognition. In: Proceedings of the 2017 International Conference on Advances in Computing, Communications and Informatics, Udipi, India, pp 13–16
35. Khan R.U, Zhang X, Alazab M, Kumar R (2019) An improved convolutional neural network model for intrusion detection in networks. In: 2019 Cybersecurity and Cyberforensics Conference (CCC), pp 74–77. <https://doi.org/10.1109/CCC.2019.000-6>
36. Ni S, Qian Q, Zhang R (2018) Malware identification using visualization images and deep learning. *Comput Security* 77:871–885
37. Vinayakumar R, Alazab M, Soman KP, Poornachandran P, Al-Nemrat A, Venkatraman S (2019) Deep learning approach for intelligent intrusion detection system. *IEEE Access* 7:41525–41550. <https://doi.org/10.1109/ACCESS.2019.2895334>
38. Mohamed S, Ejbali R (2023) Deep sarsa-based reinforcement learning approach for anomaly network intrusion detection system. *Int J Inf Security* 22(1):235–247. <https://doi.org/10.1007/s10207-022-00634-2>
39. Shojafar M, Taheri R, Pooranian Z, Javidan R, Miri A, Jararweh Y (2019) Automatic clustering of attacks in intrusion detection systems. In: 2019 IEEE/ACS 16th International Conference on Computer Systems and Applications (AICCSA), pp 1–8. <https://doi.org/10.1109/AICCSA47632.2019.9035238>. IEEE
40. Sysdig: Security for containers, Kubernetes, and clouds. <https://sysdig.com>. Accessed 12 Dec (2021)
41. Melo A.C.D (2022) The new linux ‘perf’ tools—Slides from Linux Kongress. www.linux-kongress.org/2010/slides/lk2010-perf-acme.pdf. Accessed 11 Jan
42. Lei L, Sun J, Sun K, Shenefiel C, Ma R, Wang Y, Li Q (2017) Speaker: Split-phase execution of application containers. In: Polychronakis, M., Meier, M. (eds.) Detection of intrusions and malware, and vulnerability assessment, pp 230–251. Springer, Cham https://doi.org/10.1007/978-3-319-60876-1_11
43. Forrest S, Hofmeyr SA, Somayaji A (1997) Computer immunology. *Commun ACM* 40(10):88–96. <https://doi.org/10.1145/262793.262811>
44. Tavallaei M, Bagheri E, Lu W, Ghorbani A.A (2009) A detailed analysis of the kdd cup 99 data set. In: 2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications, pp 1–6. <https://doi.org/10.1109/CISDA.2009.5356528>
45. Creech G, Hu J (2014) A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns. *IEEE Trans Comput* 63(4):807–819. <https://doi.org/10.1109/TC.2013.13>
46. Haider W, Hu J, Slay J, Turnbull BP, Xie Y (2017) Generating realistic intrusion detection system dataset based on fuzzy qualitative modeling. *J Netw Comput Appl* 87:185–192. <https://doi.org/10.1016/j.jnca.2017.03.018>
47. Hossin M, Sulaiman M.N (2015) A review on evaluation metrics for data classification evaluations. *Int J Data Min Knowl Manage Process* 5(2). <https://doi.org/10.5121/ijdkp.2015.5201>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.